Software Requirements Specification

for

TrustLens

Version 2.0

Prepared by:

<Name> ***<Reg No>***

<Branch>

<Course Code and Title>

<Assignment>

Faculty: <*Faculty Name*>

Semester: <*Semester*> Year: <*Year*>

Date Created: 24 August 2026

Structure follows IEEE Std 830-1998 and the Karl E. Wiegers SRS template (© 1999), used and modified by permission.

Table of Contents

List of Figures

Revision History

Name	Date	Reason For Changes	Version
<Name>	23 August 2026	First SRS. Requirements taken from the programme charter and rearranged into the IEEE 830 / Wiegers structure.	1.0
<Name>	24 August 2026	Rewritten to the department SRS format. Requirements renumbered by section (4.1.3.1 style) instead of REQ-n, nonfunctional requirements moved into ID / Category / Requirement tables, and the charter cross-reference tags removed from the requirement text. Appendix B now carries the ER model, the three data flow diagrams and the use case diagram. No requirement was dropped.	2.0

1. Introduction

1.1 Purpose

TrustLens is a scam-detection and evidence-preservation tool for Indian digital fraud. A person who has received a suspicious SMS, WhatsApp message, email or link can submit it and get back a verdict that says how risky it looks, how sure the system is, and which official advisory the judgement rests on. The system also stores the submitted content with a hash so it can be used as evidence later, and packages it into a report the person can take to a bank or to the police.

This document covers the whole product rather than one module. It describes release 1.0, which is the text-message path end to end, and it also describes the features planned for after that (enrichment adapters, AI assistance and analytics) so that the boundary between what gets built now and what gets built later is written down instead of assumed.

It says what the software has to do. It does not say how to build it.

1.2 Document Conventions

The IEEE 830-1998 structure and the Wiegers template were used. Requirements are numbered by the section they sit in, so requirement 4.3.3.2 is the second functional requirement of feature 4.3.

The word **shall** marks a binding requirement. **Should** is a recommendation and is never used inside a numbered requirement.

Priority is given per feature in its 4.x.1 subsection and, where a feature contains a mix, per requirement. Three levels are used:

- **High** — release 1.0 is not finished without it
- **Medium** — wanted in release 1.0, but the release could ship without it
- **Post-MVP** — specified now, built later

Every requirement here came from the programme charter, which in turn came from the project brief. Nothing was invented while moving requirements into this document. A small number of requirements rest on our own engineering judgement rather than on an instruction from anyone, and those are called out where they appear.

1.3 Intended Audience and Reading Suggestions

The document is written for the project team and the faculty reviewing it. In a real setting it would also be read by developers, testers and whoever is paying for the work.

Read it in order. Sections 1 and 2 give the context and can be read on their own. Section 3 draws the line between TrustLens and everything outside it. Section 4 is the part a developer would build from. Section 5 covers the qualities the system has to have regardless of feature. Appendix C lists what is still undecided, and it is worth reading before treating any of this as final.

1.4 Product Scope

Digital fraud in India tends to run as a staged funnel. The victim is contacted, given a pretext, pushed into either trusting or fearing the caller, asked for a credential or a payment, and then told to keep quiet about it. The damaging step is almost always performed by the victim: entering a UPI PIN, scanning a QR code, installing an APK, or paying a "fine" to somebody claiming to be a police officer.

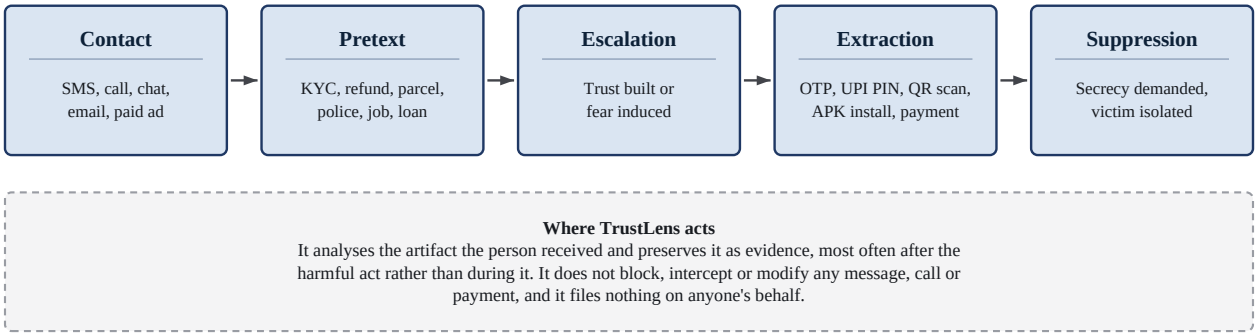


Figure 1 — The stages a typical scam moves through, and where TrustLens fits.

There is already a lot of official guidance on this from I4C, CERT-In, RBI, NPCI, SEBI and DoT. The problem is that it is not in a form software can use. It sits in advisory PDFs and press releases, written for people to read, and it goes out of date. TrustLens is an attempt to close the gap between that published guidance and a decision somebody can act on at the moment they need it.

What the product should achieve:

1. Give a verdict on submitted content that keeps **risk** and **confidence** as two separate things and shows the evidence for both.
2. Make every finding traceable, from the submitted content through the extracted indicators and the matched rules to the official source behind the rule.
3. Preserve the submitted evidence with integrity metadata and turn it into a report bundle.
4. Make a decision taken today reproducible months later from the stored evidence and the rule-set version that was used.

Out of scope. TrustLens does not file reports with any authority automatically. It does not issue an official determination of fraud and it does not give legal advice. It does not block, intercept or change any message, call or payment, and it does not run as a device agent or read device state. It does not let an AI model make the final decision, and it does not retrain itself on user feedback without a human approving the change. It does not try to identify or profile individual offenders.

One limitation worth stating up front. There is no labelled real-world corpus of Indian scam messages available to us, and we cannot obtain one within this project. That means precision, recall and false-positive rate cannot be measured, so no accuracy claim appears anywhere in this document. The things that *can* be proven without such a corpus, such as determinism, traceability and schema conformance, are specified in Section 5.4 with a verification method attached to each.

1.5 References

#	Document	Version / Date	Location
R1	Programme charter, from which all requirements here were taken	v1.1, 2026-08-14	`docs/00-program/PROGRAM-001-program-charter.md`
R2	Repository assessment	v1.0	`docs/00-program/BASELINE-001-repository-assessment.md`
R3	Phase 1 gate assessment	2026-08-14	`docs/00-program/GATE-001-phase-1-assessment.md`
R4	Glossary (normative vocabulary)	v1.0	`docs/00-program/glossary.md`
R5	Assumption, risk and conflict registers	v1.0-v1.1	`docs/00-program/`
R6	Source inventory, 26 sources graded, 11 verified	v1.1	`docs/01-research/RESEARCH-001-source-inventory.md`
R7	Scam taxonomy, 10 categories, 41 subcategories	v1.1	`docs/01-research/RESEARCH-002-scam-taxonomy.md`
R8	Evidence matrix, 30 starter rules graded	v1.1	`docs/01-research/RESEARCH-004-evidence-matrix.md`
R9	Research gap register, 22 open gaps	v1.1	`docs/01-research/RESEARCH-005-gap-register.md`
R10	ADR-0003, rule representation format	Accepted 2026-08-15	`adr/ADR-0003-rule-representation-format.md`
R11	Rule JSON Schema (draft 2020-12)	schema version 1.0	`knowledge/schemas/rule.schema.json`
R12	Source verification manifest	2026-08-14	`knowledge/sources/verification-manifest.json`
R13	IEEE Std 830-1998, *Recommended Practice for Software Requirements Specifications*	1998	IEEE
R14	K. E. Wiegers, *Software Requirements Specification template*	1999	Template used for this document
R15	Analysis models — ER, DFD levels 0 to 2, use case	2026-08-24	`docs/05-architecture/diagrams/`, reproduced in Appendix B

2. Overall Description

2.1 Product Perspective

TrustLens is a new, self-contained product. It does not replace anything, it is not part of a product family, and it does not extend an existing tool. There is no legacy code and no earlier deployment to work around.

It is not self-sufficient in knowledge, though. Everything it can detect comes from a curated knowledge base built out of published official guidance, and that guidance reaches the system through a governed pipeline with a human in it rather than an automatic feed. A TrustLens with an empty rule set is a working system that detects nothing.

The product boundary is drawn as a level 0 data flow diagram in Appendix B. TrustLens exchanges data with nine outside parties:

External entity	Direction	What is exchanged
Reporting user	both	Submits artifacts and corrections; gets back a verdict, an explanation and a report bundle
Analyst	both	Receives queued uncertain cases; returns adjudications
Administrator	both	Supplies configuration and retention policy; receives health, audit and metrics
Report recipient (authority, bank, portal)	outbound	Receives an access-controlled report export, never an automatic submission
Official guidance bodies (I4C, CERT-In, RBI, NPCI, SEBI, DoT)	inbound only	Published advisories, consumed one way and graded before use
Knowledge editor	both	Submits draft rules; gets back schema and lint verdicts
Knowledge approver	both	Receives review packages; returns publication decisions
URL / threat-intelligence providers *(Post-MVP)*	both	Reputation lookups and non-authoritative verdicts
AI assist provider *(Post-MVP)*	both	Isolated content prompts out; schema-checked drafts back

2.2 Product Functions

There are eight things the system does. They match the eight processes in the level 1 data flow diagram in Appendix B, so the two can be checked against each other.

- **Ingest and preserve evidence** — take the submitted artifacts, check them, hash each one and record chain-of-custody metadata
- **Normalise and extract** — convert content to a canonical form, work out the language and script, and pull out entities, indicators and negative indicators
- **Evaluate rules and score** — run a pinned rule set over the extracted evidence and compute risk, confidence, severity and evidence quality as four separate numbers
- **Compose explanation** — report what matched, what was checked and did not match, what reduced the risk, why confidence is limited, and which official source backs each finding
- **Route and adjudicate** — send uncertain cases to an analyst and record the decision with its reasoning
- **Assemble report bundle** — build an exportable package from a case, with the disclaimer attached
- **Govern the knowledge base** — grade sources, validate and lint draft rules, run the review and approval

workflow, publish and roll back rule-set versions

- **Administer, audit and observe** — authenticate users, enforce role-based access, apply the retention policy and write audit events

2.3 User Classes and Characteristics

Six user classes are expected. The first two decide whether the product succeeds; the rest are internal operators.

We should be honest that none of these have been validated. No end user, analyst, knowledge editor or administrator was available to talk to, so all six descriptions are our own inference. This is the biggest single source of requirement risk in the document.

Class	How often	Technical level	Access	What they need
Reporting user, general (P1)	Occasionally, usually in a hurry	Low. Comfortable with UPI and messaging, not technical	Own data only	Salaried, urban, about thirty seconds of patience, mildly anxious. Wants a fast plain-language verdict and one clear next step. Fails if the answer is hedged into meaninglessness or full of jargon. This is the most important class to get right.
Reporting user, high-harm (P2)	Rarely, in a crisis	Low. Inclined to defer to authority	Own data only	Retired, WhatsApp-first, targeted by coercion scams such as digital arrest. Needs a calm, unambiguous contradiction of what the scammer told them, and explicit permission to involve family. Fails if the system equivocates or adds to the panic.
Analyst (P3)	Daily	High, domain expert	Read all cases, adjudicate	Works the queue of uncertain cases. Needs the full score breakdown and needs to see what did *not* match. Fails if they cannot tell why the engine concluded what it did.
Knowledge editor (P4)	Weekly	High, domain expert	Author rules, cannot publish	Turns new advisories into rules. Needs a schema that makes a well-formed rule easy to write and a malformed one impossible. Fails if adding a rule means touching engine code.
Knowledge approver (P5)	Weekly	High	Publish, reject, roll back	Gatekeeps publication. Needs a diff, an impact analysis and regression results before approving. Fails if rules can reach production unreviewed.
Administrator (P6)	As needed	High, operational	Full configuration, no case content	Runs the platform. Needs health signals, rollback and retention controls, and should not need access to submitted content to do any of it.

Official guidance bodies and report recipients also interact with the product but are not user classes. The first are consumed one way, the second receive an export without operating the software.

2.4 Operating Environment

Element	Specification
Client	Current evergreen browsers on mobile and desktop: Chrome, Safari, Firefox, Edge. Mobile first, since the primary user is assumed to be on a phone. No native mobile app in release 1.0.
Frontend	React with TypeScript in strict mode
Backend	Java 21 with Spring Boot 3.x, built as a modular monolith

Element	Specification
AI / advanced extraction	Python with FastAPI as a separate deployable service. Deferred to the AI phase.
Database	PostgreSQL 16
Local environment	Docker and Docker Compose
CI/CD	GitHub Actions
Deployment target	Local and test environments only for now. No production deployment happens during this project, and the hosted target has not been decided.

Java 21.0.11, Maven 3.9.11 and Node 26.0.0 are installed on the development machine. Docker and PostgreSQL are not, which blocks implementation but not the specification work.

2.5 Design and Implementation Constraints

ID	Constraint
2.5.1	No automatic submission of legal or regulatory reports, to any authority, by any route.
2.5.2	No accuracy claim unless it comes from a reproducible evaluation on a described dataset.
2.5.3	The rule engine is the decision authority. AI output is advisory and never authoritative.
2.5.4	No fabricated advisories, citations, statistics, regulatory obligations or datasets.
2.5.5	Synthetic examples shall be labelled synthetic and never presented as real samples.
2.5.6	Docker and PostgreSQL are missing from the development machine, so implementation is blocked until they are installed.
2.5.7	Delivery capacity is one engineer plus AI assistance.
2.5.8	Java 21 with Spring Boot on the backend and React with TypeScript on the frontend. The Python service is deferred.
2.5.9	Rules are versioned JSON data validated by a published JSON Schema and a cross-file linter, never engine code. Adding a new scam type means writing one JSON file.
2.5.10	Rule identifiers are neutral (`TL-<domain>-<nnn>`). Source attribution is carried as data, not embedded in the key.
2.5.11	No rule may fire on a single indicator, and no rule may fire on weak indicators alone. This is enforced at load time, not by convention.
2.5.12	Severity is an ordinal from `LOW` to `CRITICAL`. Numeric risk or confidence values are not stored on a rule.
2.5.13	Three of the starter rules need evidence TrustLens cannot observe: device network state, user journey and live payment flow. They stay marked `DEFERRED` with a reason and are not published.

2.6 User Documentation

The following will ship with the software. Online material in HTML, anything meant to leave the system with a user in PDF.

Component	Audience	Content
In-product help and first-run guidance	Reporting user	What TrustLens does and does not do, what the disclaimer means, how to read a verdict
Verdict reading guide	Reporting user	How to read risk against confidence, and what `INSUFFICIENT_EVIDENCE` means

Component	Audience	Content
Report bundle README	Reporting user, report recipient	What is in the bundle, how to verify the hashes, and the statement that it is not an official determination
Analyst adjudication guide	Analyst	Queue workflow, score breakdown, how to override and record the reason
Rule authoring guide	Knowledge editor	Schema walkthrough, a worked example for each rule shape, catalogue of lint errors
Approval and rollback runbook	Knowledge approver	Diff review, impact analysis, regression evidence, rollback procedure
Operations runbook	Administrator	Configuration reference, retention controls, health checks, backup and recovery
API reference	Integrator, developer	Generated from the OpenAPI document

2.7 Assumptions and Dependencies

Assumptions we are working under:

- No end user, analyst or official-body stakeholder is available, so every persona and journey in this document is our own inference. This one is load-bearing and we have low confidence in it.
- The deployment is single tenant. Multi-tenancy is a future concern.
- Release 1.0 is English first. The architecture is multilingual but the content is not.
- No real user data is processed during the project. All examples are synthetic and labelled.
- There is no budget for paid threat-intelligence providers, so those adapters target free tiers or are stubbed.
- Users submit content after the fact, usually on a phone, rather than during a live attack.
- TrustLens holds no regulatory registration, official status or relationship with any body whose guidance it cites.
- Submitted content routinely contains live sensitive data such as OTPs, account numbers and VPAs, and has to be treated as sensitive from the moment it arrives, before anything is classified.
- The latency budget is seconds, not milliseconds.
- Default evidence retention is 90 days. That is a placeholder, not a policy decision.
- India's Digital Personal Data Protection Act 2023 is *likely* to apply, but this has not been legally verified and no compliance claim is being made here.
- During development, rule authoring and approval are done by the same person. The separation of duties is built and tested in the system even though it is not enforced organisationally.

Things the project depends on:

Dependency	What it is	Why it matters
Official-source retrieval is partly manual	`i4c.mha.gov.in`, `pib.gov.in` and `npci.org.in` block automated retrieval; `cert-in.org.in`, `niti.gov.in` and `rbi.org.in` allow it. We tested this rather than assuming it.	Any advisory-ingestion feature has to assume a human retrieval step. Full automation is not achievable.
Knowledge base completeness	26 sources graded and 11 verified; 30 starter rules graded, of which 18 are both evidenced and implementable.	Detection coverage at release is limited by the evidence available, not by what the engine can do.

Dependency	What it is	Why it matters
Negative-indicator library	The research package gives us 12 positive indicator families and zero suppressive signals.	Without negative indicators the false-positive problem cannot be solved. This is the highest-priority piece of knowledge work outstanding.
No labelled corpus	None exists and none can be obtained within the project.	Accuracy cannot be measured, which is why Section 5.4 is limited to provable attributes.
Docker and PostgreSQL	Not installed on the development machine.	Blocks implementation entirely.
Third-party providers *(Post-MVP)*	URL and threat-intelligence services, and an AI provider.	The core path has to finish even with any one of them unavailable.
OCR engine *(Post-MVP)*	Needed for screenshot submission. We found no guidance on OCR quality for Indian scripts.	Evidence-quality scoring for OCR output is unspecified so far.

3. External Interface Requirements

3.1 User Interfaces

One responsive web application, mobile first, with navigation that changes by role. There is no separate installable client in release 1.0.

Screens:

Screen	Who uses it	Purpose
Submit	P1, P2	Paste or upload content, optionally add sender and channel context, submit
Verdict	P1, P2	Plain-language outcome, risk and confidence shown separately, next steps, a "why" expander
Evidence detail	P1, P3	Indicators found, rules matched, rules checked and not matched, source citations
Correction	P1	Fix an extracted entity and re-run the evaluation
Case	P1, P3	Submissions, findings, notes and bundle export for one incident
Review queue	P3	Uncertain cases with the full score breakdown and adjudication controls
Rule authoring	P4	Draft a rule, run validation, read the lint errors
Approval	P5	Diff, impact analysis, regression results, publish / reject / roll back
Administration	P6	Configuration, thresholds, retention, feature flags, health

Rules that apply to every screen:

- WCAG 2.2 AA on all release 1.0 journeys, checked by automated and manual audit.
- Risk and confidence are never merged into one number, one bar or one colour anywhere in the UI.
- Every verdict view carries the "not an official determination" disclaimer without the user having to click anything to see it.
- The plain-language explanation is the default view. The technical breakdown is one action away and is never what P1 or P2 land on.
- Supported languages are stated in the interface. Unsupported input is flagged rather than quietly returning a weak result.
- Error messages say what went wrong and what to do next, and never include raw submitted content, secrets or internal identifiers.

Layout, interaction and visual design are out of scope for this document and belong in a separate UI specification.

3.2 Hardware Interfaces

None. TrustLens is a browser-delivered web application with no direct hardware dependency. It does not talk to device sensors, SIM or telephony hardware, card readers or any physical peripheral. Camera and file access for screenshot upload go through standard browser APIs, so the software has no knowledge of the underlying device.

This is deliberate rather than an omission. Device-agent behaviour is explicitly out of scope, and constraint 2.5.13 records the three rules that consequently cannot be implemented.

3.3 Software Interfaces

#	Component	Version	In	Out	Purpose
3.3.1	PostgreSQL	16	Evidence records, cases, findings, pinned analyses, audit events	The same on read	Primary datastore, reached through a connection pool over TLS
3.3.2	Rule JSON Schema	schema version 1.0	Rule documents at load time	Validation verdict	The single authority on rule structure. The Java loader validates against the schema file itself, not a reimplementation of it.
3.3.3	Cross-file rule linter	—	Rule set, indicator registry, taxonomy, verification manifest	Lint verdict naming the layer that caught the problem	Enforces the constraints a single-document schema cannot express. Runs in CI before any rule can be published.
3.3.4	Source verification manifest	2026-08-14	Source identifiers	Verification status and grade	A rule cannot claim a grade the manifest contradicts.
3.3.5	OCR engine *(Post-MVP)*	Not selected	Image artifacts	Extracted text with per-block confidence	Screenshot submission. The confidence value feeds evidence quality.
3.3.6	URL / threat-intelligence providers *(Post-MVP)*	Provider-agnostic adapter	URL, domain	Reputation verdict, labelled non-authoritative	Enrichment. Several providers behind one interface.
3.3.7	AI assist provider *(Post-MVP)*	Provider-agnostic adapter	Isolated content prompt	Schema-constrained JSON	Extraction help and draft rule suggestions. Advisory only.
3.3.8	Identity provider	Not selected	Credentials or a federated assertion	Authenticated principal with roles	Authentication and role assignment.

The rule set, indicator registry and scam taxonomy are shared between the knowledge-governance side and the detection side. They are shared as immutable versioned snapshots rather than a mutable store both sides write to, so that publishing a new version cannot retroactively change an analysis that has already finished.

3.4 Communications Interfaces

Interface	Specification
Client to server	HTTPS only, TLS 1.3, versioned REST over JSON
API style	Versioned REST described by an OpenAPI document, from which the published reference is generated. Asynchronous messaging only where the workload actually justifies it.
Outbound enrichment *(Post-MVP)*	HTTPS to provider endpoints with a per-provider timeout and a circuit breaker. A provider failure degrades the result and never fails the submission.

Interface	Specification
Report export	Authenticated, access-controlled download over HTTPS. There is no outbound transmission of a report to any authority by any protocol, and this is enforced by the capability not existing rather than by a configuration flag.
Email / SMS	TrustLens sends no SMS and no scam-related notification traffic. Transactional email, if it is added, is limited to account and access functions.
Correlation	Every request carries a correlation identifier that is propagated through logs and audit events.
Data formats	JSON for API payloads, UTF-8 throughout. The original byte sequence of submitted content is kept unmodified alongside the normalised form.
Rate limiting	Public endpoints are rate-limited and protected against abuse (Post-MVP).

4. System Features

Ten features. Each one states its priority, the stimulus and response pairs that exercise it, and its numbered functional requirements.

4.1 Submission and Evidence Preservation

4.1.1 Description and Priority

Takes content from a reporting user, checks it, and preserves each item with integrity metadata before any analysis runs. Preservation happens first on purpose, so that the integrity of the evidence does not depend on what the analysis later concludes. High priority: nothing else in the product works without it.

4.1.2 Stimulus/Response Sequences

#	Stimulus	Response
S1	User pastes SMS or chat text and submits	System accepts the artifact, hashes it, records the capture time and chain-of-custody metadata, creates a submission and starts the analysis
S2	User submits a URL along with the message text	System groups both artifacts into one submission and one case
S3	User uploads a screenshot that is too large or of an unsupported type	System rejects the upload before processing it, states the limit and the accepted types, and keeps the rest of the submission
S4	User adds sender, channel and description	System stores the context and passes it to extraction marked as user-supplied, so it stays distinguishable from anything derived from the content

4.1.3 Functional Requirements

4.1.3.1: The system shall accept free-text submission of SMS, WhatsApp or other chat message bodies.

4.1.3.2: The system shall accept URL submission.

4.1.3.3: The system shall accept email content including headers.

4.1.3.4: The system shall accept screenshot and image upload. *(Post-MVP)*

4.1.3.5: The system shall group multiple artifacts into a single submission and a single case.

4.1.3.6: The system shall validate media type, size and structure before processing, and reject unsafe uploads without processing them. *(Post-MVP)*

4.1.3.7: The system shall accept optional user-supplied context consisting of sender, channel and description.

4.1.3.8: The system shall compute and store a cryptographic hash of every evidence item on ingest, together with its chain-of-custody metadata.

4.1.3.9: The system shall keep the original submitted content unmodified alongside any normalised form derived from it.

4.2 Normalisation and Evidence Extraction

4.2.1 Description and Priority

Turns preserved artifacts into a canonical form and derives the structured evidence that detection works on: entities, indicators, and negative indicators. Extraction aims for high recall and attaches no score to anything, because an observation is not yet a finding. High priority.

4.2.2 Stimulus/Response Sequences

#	Stimulus	Response
S1	An artifact enters extraction	System normalises it deterministically, identifies the language and script, and extracts entities and indicators
S2	Content contains "Your OTP is 452901. Never share it with anyone."	System extracts both the OTP mention and the self-protective warning, the second as a negative indicator, so the message is not treated as a credential request
S3	Content is in an unsupported language or script	System flags the input as unsupported rather than quietly returning a weak result
S4	User corrects a misextracted phone number	System records the correction as user-supplied, re-runs the evaluation and produces a new analysis without discarding the original

4.2.3 Functional Requirements

4.2.3.1: The system shall normalise content to a canonical form deterministically, so that the same input always produces the same canonical output.

4.2.3.2: The system shall identify the language and script of submitted content.

4.2.3.3: The system shall handle code-mixed and transliterated Indian-language input. *(Post-MVP)*

4.2.3.4: The system shall extract text from screenshots by OCR, recording a per-block confidence value that contributes to evidence quality. *(Post-MVP)*

4.2.3.5: The system shall extract entities of at least these types: URL, phone number, UPI VPA, monetary amount, organisation name, application name and account reference.

4.2.3.6: The system shall extract indicators across the defined indicator families, and no extracted indicator shall carry a score.

4.2.3.7: The system shall detect negative indicators that suppress a rule or reduce risk.

4.2.3.8: The system shall let a user correct an extraction error and re-evaluate the submission, keeping both the original and the corrected extraction.

4.3 Rule-Based Detection and Scoring

4.3.1 Description and Priority

The core of the product. Takes the extracted evidence, runs a pinned versioned rule set over it, and produces risk, confidence, severity and evidence quality as four separate values. Rules are composite by construction, so no single indicator on its own ever produces a finding.

High priority, and the highest-risk feature in the product. Both of its failure modes damage trust: missing a real attack, and flagging a bank's own anti-fraud message as a scam.

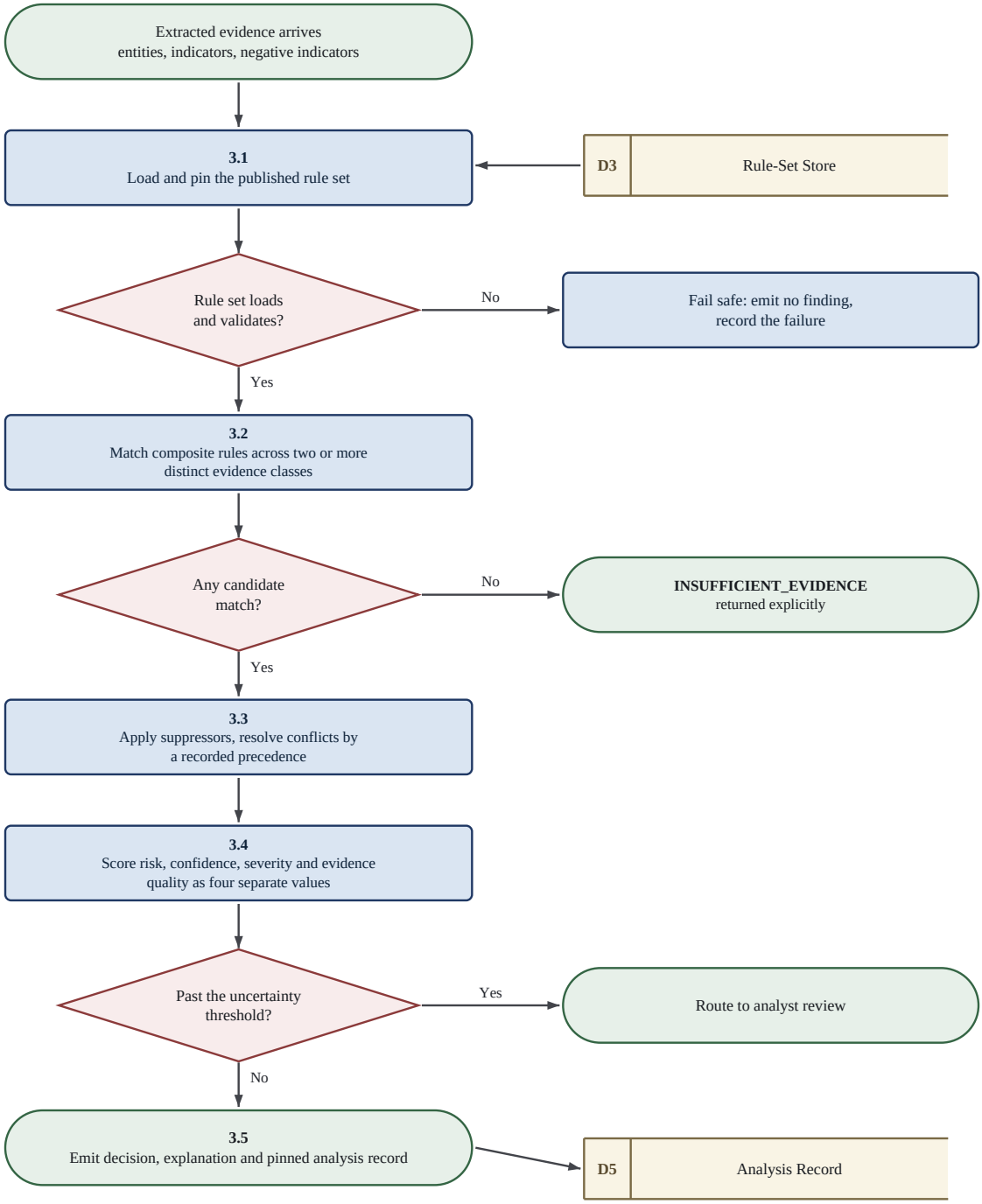


Figure 2 — How an evaluation runs, including both of the fail-safe exits. The order matters: suppressors are applied before scoring, not after.

4.3.2 Stimulus/Response Sequences

#	Stimulus	Response

#	Stimulus	Response
S1	Extracted evidence arrives for evaluation	System loads the published rule set, pins its version to this evaluation, evaluates all rules and records the result
S2	A composite rule's triggers are met across two or more evidence classes	System produces a finding with its contributing indicators and its score contribution
S3	A suppression rule's condition is met	System suppresses the affected finding and records the suppression as an explicit outcome rather than as an absence
S4	Positive and negative indicators compete	System resolves the conflict by the defined precedence and records which evidence won
S5	Evidence is too thin to conclude anything	System returns `INSUFFICIENT_EVIDENCE` instead of a low-risk verdict
S6	An evaluation goes past the configured uncertainty threshold	System routes the case to human review
S7	A malformed rule, an evaluation timeout or partial evidence	System fails safe: it emits no finding that depends on the incomplete evaluation, and records the failure
S8	An operator replays a six-month-old evaluation	System re-runs it with the pinned rule-set version and configuration and reproduces the original result exactly

4.3.3 Functional Requirements

4.3.3.1: The system shall evaluate rules deterministically against the extracted evidence.

4.3.3.2: The system shall support composite rules that require combinations of indicators spanning at least two distinct evidence classes.

4.3.3.3: The system shall support suppression rules, exceptions and exclusions.

4.3.3.4: The system shall compute risk, confidence, severity and evidence quality as separate quantities and shall never combine them into a single value, in the pipeline or in the interface.

4.3.3.5: The system shall handle correlated signals without double counting their contribution. *(Post-MVP)*

4.3.3.6: The system shall return an explicit `INSUFFICIENT_EVIDENCE` outcome when the evidence does not support a conclusion.

4.3.3.7: The system shall route evaluations past a configured uncertainty threshold to human review.

4.3.3.8: The system shall pin the rule-set version to every evaluation.

4.3.3.9: The system shall replay a historical evaluation and reproduce its result exactly.

4.3.3.10: The system shall resolve conflicts between competing positive and negative indicators by a defined and recorded precedence.

4.3.3.11: The system shall fail safe on a malformed rule, an evaluation timeout or partial evidence, emitting no finding that depends on the incomplete evaluation.

4.4 Explanation of Findings

4.4.1 Description and Priority

Turns an evaluation into something a person can check. The requirement is not that the system explains *that* it decided something, but that it breaks down *why*, including what it considered and rejected and why it is not

more confident. High priority.

4.4.2 Stimulus/Response Sequences

#	Stimulus	Response
S1	A verdict is displayed	System shows a plain-language explanation with risk and confidence separately, plus an optional technical view
S2	User opens the detail view	System lists matched rules with their contributing indicators, rules considered but not matched with the reason, negative evidence that reduced risk, and the full score breakdown
S3	Confidence is limited	System says why, and what extra context would raise it
S4	A finding rests on a rule	System shows the official source behind that rule, with its provenance grade

4.4.3 Functional Requirements

4.4.3.1: The system shall report which rules matched, together with their contributing indicators.

4.4.3.2: The system shall report which rules were considered but did not match, and why.

4.4.3.3: The system shall report negative evidence that reduced the risk.

4.4.3.4: The system shall provide a full score breakdown for each computed component.

4.4.3.5: The system shall state why confidence is limited and what context is missing.

4.4.3.6: The system shall surface the source references behind each matched rule, including each source's provenance grade.

4.4.3.7: The system shall present a plain-language explanation with an optional technical detail view.

4.4.3.8: The system shall describe how an analyst can independently verify a conclusion. *(Post-MVP)*

4.5 Analyst Review and Adjudication

4.5.1 Description and Priority

Human judgement on the cases the engine is not confident about. Feedback is captured but never triggers automatic learning: an adjudication changes this case, not the rule set. Medium priority.

4.5.2 Stimulus/Response Sequences

#	Stimulus	Response
S1	A case goes past the uncertainty threshold	System puts it in the review queue with the full breakdown and what did not match
S2	Analyst agrees, disagrees or overrides	System records the adjudication with its reasoning, writes an audit event and updates the case
S3	User submits feedback on a finding	System stores the feedback and makes no automatic change to any rule or threshold

4.5.3 Functional Requirements

4.5.3.1: The system shall present queued cases to an analyst with the full score breakdown and the rules that did not match.

4.5.3.2: The system shall support analyst adjudication with a recorded reason, and write an audit record for every override.

4.5.3.3: The system shall capture user feedback on findings without triggering any automatic change to rules, thresholds or scoring. *(Post-MVP)*

4.6 Case Management and Report Bundle

4.6.1 Description and Priority

Groups everything about one incident and produces the artifact the user actually leaves with. The bundle is only worth anything if it is reproducible, so the same case has to produce an identical bundle later. High priority.

4.6.2 Stimulus/Response Sequences

#	Stimulus	Response
S1	User requests a report for a case	System assembles the evidence with its hashes, the findings, the explanations, the source citations and the disclaimer into a structured bundle
S2	User exports the bundle	System delivers it through an authenticated, access-controlled download and records the export in the audit log
S3	The same bundle is regenerated six weeks later	System reproduces an identical bundle from the stored evidence and the pinned analysis
S4	A recipient opens the bundle	The disclaimer saying this is not an official determination is present and prominent

4.6.3 Functional Requirements

4.6.3.1: The system shall create and manage cases that group submissions, findings and analyst notes.

4.6.3.2: The system shall generate a structured report bundle conforming to the defined report contract.

4.6.3.3: The system shall reproduce an identical report bundle from the stored evidence and the pinned analysis data.

4.6.3.4: The system shall provide report export through an authenticated, access-controlled channel, and shall not transmit a report to any authority automatically.

4.6.3.5: The system shall carry a prominent disclaimer in every report bundle stating that it is not an official determination of fraud.

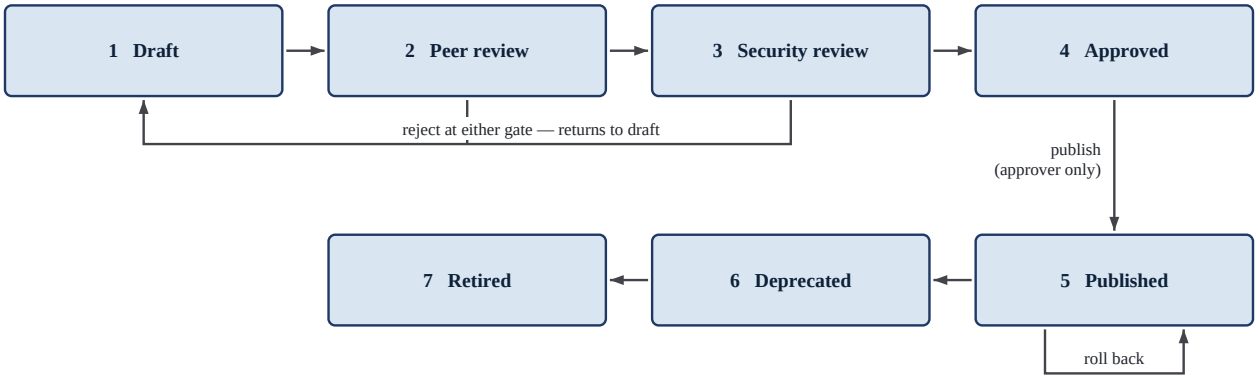
4.7 Knowledge Base Governance

4.7.1 Description and Priority

How knowledge gets into the product. This is the feature that lets a new scam type be added without touching engine code, and the feature that stops a rule claiming evidence it does not have. High priority.

4.7.2 Stimulus/Response Sequences

#	Stimulus	Response
S1	Editor submits a rule that would fire on one weak indicator	System rejects it at load time and names the constraint and the layer that caught it
S2	Editor submits a rule citing a source graded `RETRIEVAL_FAILED`	System rejects the claimed provenance grade
S3	Editor submits a valid draft rule	System validates it against the schema, runs the cross-file lint and produces a review package with a diff, an impact analysis and regression results
S4	Approver publishes a rule set	System creates a new immutable rule-set version; evaluations already in flight keep the version they pinned
S5	A published rule set turns out to be harmful	Approver rolls back to a previous version and no historical evaluation changes
S6	A new scam type is added	Editor writes one rule document and no engine code changes



An editor authors and submits (states 1 to 3); only an approver may publish, reject or roll back. Rollback restores a prior published rule-set version and changes no completed evaluation, because every analysis pins the version it used.

Figure 3 — The rule lifecycle, with the rejection path and the rollback. Publication is the only transition an editor cannot perform.

4.7.3 Functional Requirements

4.7.3.1: The system shall store rules as versioned, schema-validated data and shall not express rule logic as engine code.

4.7.3.2: The system shall reject any rule that fails schema validation or cross-file lint at load time.

4.7.3.3: The system shall maintain a versioned scam taxonomy independently of engine code.

4.7.3.4: The system shall support the rule lifecycle: draft, peer review, security review, approve, publish, deprecate, retire.

4.7.3.5: The system shall require at least one source reference carrying a provenance grade on every non-heuristic rule, and shall require any rule without such a source to be labelled heuristic.

4.7.3.6: The system shall reject a rule whose claimed source grade contradicts the source verification manifest.

4.7.3.7: The system shall support rollback to a previously published rule-set version without altering any completed evaluation. *(Post-MVP)*

4.7.3.8: The system shall perform an impact analysis when a rule, taxonomy term or source changes. *(Post-MVP)*

4.7.3.9: The system shall allow a new scam type to be added through data and configuration alone, with no change to engine code.

4.8 Identity, Access and Audit

4.8.1 Description and Priority

Establishes who is allowed to do what, and makes the security- and knowledge-relevant actions permanently reviewable. High priority.

4.8.2 Stimulus/Response Sequences

#	Stimulus	Response
S1	A user authenticates	System establishes a principal with roles and grants only the permitted capabilities
S2	An editor tries to publish a rule set	System denies it; publication needs the approver role
S3	A security- or knowledge-relevant action happens	System writes an immutable audit event with actor, action, target, timestamp and correlation identifier
S4	A user asks to export or delete their own data	System fulfils the request within the retention policy and records audit evidence that it did so

4.8.3 Functional Requirements

4.8.3.1: The system shall authenticate users.

4.8.3.2: The system shall enforce role-based access control across the reporting user, analyst, knowledge editor, knowledge approver and administrator roles.

4.8.3.3: The system shall write immutable audit events for security- and knowledge-relevant actions.

4.8.3.4: The system shall support data export and deletion requests with audit evidence that they were fulfilled.

4.8.3.5: The system shall apply the configured retention class to submitted content and shall not keep it beyond that class.

4.9 Enrichment and AI Assistance

4.9.1 Description and Priority

Two optional capabilities behind feature flags. Both are advisory. Neither may decide an outcome, and the system has to work fully with both switched off.

Post-MVP, so low priority for release 1.0, but it carries the highest risk in the document, because this is where a model could quietly end up making the decision if the boundary is not enforced.

4.9.2 Stimulus/Response Sequences

#	Stimulus	Response
S1	A submission contains a URL and enrichment is on	System queries the provider through the adapter and labels the returned verdict as external and non-authoritative
S2	The provider times out or errors	System finishes the analysis without it and records the degradation
S3	AI assistance is on	System sends the content as isolated data, quarantined from the model instructions
S4	The model returns output that does not fit the schema	System rejects the output and carries on deterministically
S5	The model suggests a new rule	System stores it as a draft that needs human approval; it cannot reach a published set without one

4.9.3 Functional Requirements

4.9.3.1: The system shall integrate URL and threat-intelligence adapters behind a provider-agnostic interface.

4.9.3.2: The system shall operate fully when any single external provider is unavailable.

4.9.3.3: The system shall gate all AI-assisted capability behind feature flags.

4.9.3.4: The system shall validate AI output against strict schemas and reject anything that does not conform.

4.9.3.5: The system shall label model-derived observations distinctly from deterministic findings, in storage, in API responses and in the interface.

4.9.3.6: The system shall require human approval before any AI-suggested rule is published.

4.9.3.7: The system shall degrade to deterministic-only operation when AI assistance is unavailable, with the core path unaffected.

4.9.3.8: The system shall isolate submitted content from model instructions so that content cannot alter model behaviour.

4.10 Analytics

4.10.1 Description and Priority

Operational and knowledge-quality reporting. It reports what the system did. It never makes a claim about how accurate the system is. Post-MVP, low priority.

4.10.2 Stimulus/Response Sequences

#	Stimulus	Response
S1	Approver reviews knowledge quality	System reports rule usage, coverage and contribution
S2	Analyst reviews adjudication history	System reports adjudicated false positives and negatives by category, scoped to the dataset they were measured on
S3	Administrator checks operations	System reports volume, latency and review-queue depth

4.10.3 Functional Requirements

4.10.3.1: The system shall report rule usage, coverage and contribution.

4.10.3.2: The system shall report adjudicated false positives and negatives by category, and shall state the dataset and its limitations alongside any such figure.

4.10.3.3: The system shall report operational quality including volume, latency and review-queue depth.

4.10.3.4: The system shall rate-limit public endpoints and protect them against abuse.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

ID	Category	Requirement
5.1.1	Performance	Analysis of a text-only submission shall complete in under 2 seconds at the 95th percentile.
5.1.2	Performance	Analysis of a screenshot submission that needs OCR shall complete in under 10 seconds at the 95th percentile. *(Post-MVP)*
5.1.3	Reliability	Rule-set load and validation shall finish before the system accepts traffic. A rule set that fails validation shall stop startup rather than degrade at runtime.

The budget is in seconds rather than milliseconds because people submit content after an incident, not during a live attack. Both latency numbers are our own estimates rather than the result of a user study, and they are the first figures we would revisit if a real user became available. We have put numbers on them anyway, because a performance requirement nobody can measure is not a requirement.

5.2 Safety Requirements

These are about harm to the user, which for this product means acting on a verdict that is wrong or that they have misread.

ID	Category	Requirement
5.2.1	Safety	The system shall not present a verdict in a way that implies an official determination of fraud.
5.2.2	Safety	The system shall not present recommendations as legal advice. Recommended actions shall be safety actions only.
5.2.3	Safety	The system shall route ambiguous or weakly supported cases to review, or return `INSUFFICIENT_EVIDENCE`, rather than presenting an uncertain conclusion as a certain one.
5.2.4	Safety	For coercion scenarios such as digital arrest, the explanation shall plainly contradict what the scammer claimed, and shall not amplify urgency or fear.
5.2.5	Safety	The system shall not block, intercept or modify any message, call or payment.
5.2.6	Safety	Where the knowledge base cannot support a conclusion in the submitted language, the system shall say so explicitly rather than returning a weak result that reads as safety.

5.3 Security Requirements

ID	Category	Requirement
5.3.1	Security	All data shall be encrypted in transit using TLS 1.3 and at rest using AES-256.
5.3.2	Privacy	Logs shall contain no secrets, no personally identifiable information and no raw submitted evidence. This shall be verified by automated scanning.
5.3.3	Privacy	Submitted content shall be treated as sensitive from the moment it is ingested, before any classification decision is made about it.
5.3.4	Auditability	Security- and knowledge-relevant actions shall be recorded immutably, covering 100% of the defined event set.
5.3.5	Access control	Access to case content shall be restricted by role. The administrator role shall be able to operate the platform without access to submitted content.

ID	Category	Requirement
5.3.6	Security	Uploaded files shall be validated and handled so that a malicious upload cannot execute or escape its processing context.
5.3.7	Security	Submitted content shall never be interpretable as an instruction by any downstream component, including AI-assisted ones.
5.3.8	Privacy	The system shall not identify or profile individual suspected offenders.

On privacy: we have not had a lawyer look at whether India's Digital Personal Data Protection Act 2023 applies to TrustLens, and this document makes no compliance claim. The requirements above are engineering practice, not a legal opinion.

5.4 Software Quality Attributes

This section is split into what can be proven and what cannot. Nothing in the first table needs real-world data. Nothing in the second may be claimed until such data exists.

5.4.1 Provable

ID	Attribute	Requirement	Target	Verified by
5.4.1.1	Determinism	Identical evidence, rule-set version and configuration shall produce identical output.	100% of golden-case replays	Automated replay suite
5.4.1.2	Explainability	No finding shall exist without a complete evidence, rule and source trace.	100% of findings	Property test
5.4.1.3	Extensibility	A new scam type shall be addable with no engine code change.	0 engine lines changed	Integration test
5.4.1.4	Correctness of knowledge	Every published rule shall pass schema validation and cross-file lint.	100%	CI gate
5.4.1.5	Traceability	Every non-heuristic published rule shall carry a graded source reference, and every rule without one shall be labelled heuristic.	100%	CI gate
5.4.1.6	Accessibility	Release 1.0 journeys shall meet WCAG 2.2 AA.	Pass	Automated and manual audit
5.4.1.7	Testability	Automated gates shall exist at unit, schema, property, integration, contract and end-to-end level.	CI-enforced	CI
5.4.1.8	Reproducibility	A clean clone shall build and run without manual intervention.	Green from scratch	CI
5.4.1.9	Observability	All services shall emit structured logs with correlation identifiers, plus metrics and health checks.	All services	Operational review
5.4.1.10	Maintainability	A new scam type shall be addable by a knowledge editor without a developer being involved.	Proven by test	Integration test
5.4.1.11	Availability of core path	The core analysis path shall complete when enrichment or AI assistance is unavailable.	Core path unaffected	Fault-injection test

5.4.2 Not provable within this project

Precision, recall, false-positive rate, false-negative rate, calibration and abstention quality cannot be claimed. There is no labelled real-world corpus and we cannot get one. A synthetic corpus is useful for determinism and

regression detection and nothing else. Any figure produced against synthetic data shall be reported with its dataset scope and limitations attached, and shall never be presented as a general accuracy claim.

5.5 Business Rules

ID	Rule
5.5.1	The rule engine decides. AI assists interpretation and extraction; it never overrides deterministic evidence and is never the final authority.
5.5.2	Risk and confidence are separate quantities and are never collapsed into one number, anywhere.
5.5.3	A new scam type is added through data, not code.
5.5.4	No rule fires on a single indicator. Every published rule needs a combination spanning at least two distinct evidence classes.
5.5.5	Publication takes two roles. An editor may author and submit; only an approver may publish, reject or roll back. During development one person may hold both roles, but the system enforces the separation regardless.
5.5.6	A rule may not claim evidence it does not have. A rule whose source grade contradicts the verification manifest cannot be published, and an unsupported or heuristic rule cannot enter the published set at all.
5.5.7	TrustLens files nothing on anyone's behalf. Report bundles leave only through a user-initiated, access-controlled export.
5.5.8	Feedback never auto-trains. No user or analyst action changes a rule, a threshold or a score without human adjudication and the approval workflow.
5.5.9	Uncertainty is shown, not hidden. `INSUFFICIENT_EVIDENCE` is a first-class outcome, not a failure state.
5.5.10	Synthetic content is labelled synthetic and is never presented as a real sample.
5.5.11	Administrators operate the platform; they do not read cases. Platform administration does not carry access to submitted content.

6. Other Requirements

Database. Evidence, cases, findings, pinned analyses and audit events shall be stored so that an evaluation's inputs and rule-set version stay retrievable for replay for the full retention period, audit events are append-only, and deletion under the retention policy or at a user's request removes the submitted content while leaving audit evidence that the deletion happened.

Internationalisation. All user-facing text shall be externalised for translation from the first release, and language and script shall be first-class fields in the rule schema rather than something retrofitted later, even though release 1.0 ships English content only. The system shall state which languages it supports and flag unsupported input. Which Indian languages make it into release 1.0 has not been decided.

Legal. TrustLens holds no regulatory registration, official status or relationship with any body whose guidance it cites. Every report bundle and every verdict view shall carry the "not an official determination" disclaimer. No statement in the product shall assert a regulatory obligation that has not been checked against a primary source.

Reuse. The rule schema, indicator registry, scam taxonomy and source verification manifest are designed as portable data artifacts, independent of the engine that reads them, so that a future device-side or partner component could reuse them without reimplementation.

Deliberately not in this document, following IEEE 830 and the course notes: project cost, delivery schedule, staffing and reporting procedures; design solutions such as module partitioning and data-structure choice; and product assurance procedures such as QA, configuration management and verification plans. Those belong in the project roadmap, the architecture decision records and the test strategy.

Appendix A: Glossary

The terms below are normative. Where they appear in TrustLens code, artifacts or interface, they mean exactly this. The four decision quantities are kept separate on purpose, because the whole design rests on not collapsing them into one number.

Decision quantities

Term	Definition	What it is not
Severity	How much harm the scam pattern would cause *if the finding is right*. A property of the scam class, mostly fixed per rule. Ordinal: `LOW`, `MEDIUM`, `HIGH`, `CRITICAL`.	Not a measure of whether the pattern is actually present.
Risk	Computed exposure for *this particular submission*. A function of severity and the strength of the matched evidence. Bounded, decomposable, reproducible.	Not a probability, and not "how sure we are".
Confidence	How much the system trusts its own analysis of this submission: extraction quality, corroboration across independent indicators, completeness of context.	Not risk. A confident finding can be low risk.
Evidence quality	How reliable the *inputs* were: OCR fidelity, truncation, known sender, whether enrichment succeeded. Feeds confidence.	Not the reliability of the rule's source.
Signal strength	What a single indicator or rule match contributes before aggregation.	Not the final score.
Trust	The reliability weight of a knowledge *source* or provider. A property of the source.	Not confidence in the finding.

Pipeline and domain terms

Term	Definition
Submission	One act of sending content for analysis. Contains one or more artifacts.
Artifact	A single piece of submitted content: a text body, a URL, an image, an email source. Immutable once stored.
Normalisation	Deterministic conversion of an artifact to canonical form without discarding the original.
Extraction	Deriving entities and indicators from normalised content.
Entity	A concrete identifiable thing found in the content: URL, phone number, UPI VPA, amount, organisation, app name, account reference.
Indicator	An observed signal belonging to an indicator family, for example `CREDENTIAL_REQUEST` or `SECRECY_DEMAND`. Carries no score.
Negative indicator	An observed signal that reduces risk or suppresses a rule, for example "never share this OTP".
Rule	A versioned, declarative, source-referenced statement that a named combination of indicators is a recognised scam pattern. Stored as validated data, not code.
Rule set	A versioned collection of rules published together. Evaluations pin the version.
Evaluation	One deterministic run of a rule set against one submission's extracted evidence.
Finding	A single rule match, with its contributing indicators, score contribution and source references.
Decision	The overall outcome across all findings, including `INSUFFICIENT_EVIDENCE`.
Explanation	The account of a decision: what matched, what did not, what reduced risk, why confidence is limited, how to verify.

Term	Definition
Case	A durable container grouping the submissions, evidence, findings, notes and reports for one incident.
Evidence item	An artifact plus its integrity metadata: hash, capture timestamp, chain-of-custody record.
Report bundle	A reproducible, exportable package assembled from a case. Assists reporting; not an official determination.
Adjudication	An analyst's recorded judgement on a finding or case, including the reasoning.
Provenance	The recorded origin of a knowledge item: which source, which advisory, retrieved when, verified how.
Replay	Re-running a historical evaluation with its pinned rule-set version to reproduce the original result exactly.

Evidence and provenance grades

Grade	Meaning
`PRIMARY_VERIFIED`	The issuing body's own document was retrieved and the specific claim was located inside it.
`PRIMARY_CITED_UNVERIFIED`	A specific primary document is cited but has not been retrieved and checked.
`INDEX_ONLY`	The citation resolves to a listing or index page, not to a document that substantiates the claim. Not sufficient on its own.
`SECONDARY`	The claim comes from a synthesis or a commentary about a primary source.
`HEURISTIC`	An engineering judgement with no source claim. Has to be labelled as such.
`SYNTHETIC`	Example content written for testing. Never presented as a real sample.

India-specific terms

Term	Definition
UPI	Unified Payments Interface, India's real-time retail payment system, operated by NPCI.
UPI PIN	The secret that authorises *sending* money over UPI. Receiving money never needs it, which is the basis of several rules.
VPA	Virtual Payment Address, the `name@bank` identifier used to address UPI payments.
OTP	One-Time Password. Note the difference between a message *delivering* one and a message *requesting* one.
KYC	Know Your Customer, regulated identity verification. A frequent scam pretext.
Digital arrest	A coercion scam that impersonates law enforcement, alleges criminal involvement, demands secrecy and extracts payment under threat of arrest.
Smishing / Vishing	Phishing carried out over SMS and over a voice call respectively.
USSD	Telephony short codes such as `*21#`, abused to silently enable call forwarding.
APK / sideloading	Installing Android apps from outside the official store. A malware delivery route.

Organisations

Abbrev.	Body	Authority level
I4C	Indian Cybercrime Coordination Centre, Ministry of Home Affairs	Official (government)

Abbrev.	Body	Authority level
NCRP	National Cyber Crime Reporting Portal	Official (government)
CERT-In	Indian Computer Emergency Response Team	Official (government)
RBI	Reserve Bank of India	Official (regulator)
NPCI	National Payments Corporation of India	Official (payment system operator)
SEBI	Securities and Exchange Board of India	Official (regulator)
DoT	Department of Telecommunications	Official (government)
Sanchar Saathi / Chakshu	DoT citizen portal for reporting suspected fraud communication	Official (government)
Commercial organisations (banks, platform companies)	—	Not authorities. Corroborating industry guidance only.

Document and programme terms

Term	Definition
SRS	This document. The statement of what developers are to implement.
DDI-style composite rule	A rule that needs two or more indicators from different evidence classes before it fires.
MVP	Release 1.0 scope: a verifiable verdict on a text message, including evidence preservation, the case and the report bundle.
DFD	Data Flow Diagram, the analysis model used in Appendix B.
ER model	Entity Relationship model, the data model used in Appendix B.

Appendix B: Analysis Models

Four models are given here: the entity relationship model, the data flow diagrams at levels 0, 1 and 2, and the use case diagram. Together they cover the data the system stores, how that data moves through it, and what each actor can ask it to do.

The DFD notation is DeMarco and Yourdon: an oval is a process, a rectangle is an external entity and a cylinder is a data store. Level 0 contains no data store, by convention. Source files for all of these live in `docs/05-architecture/diagrams/`.

B.1 ER Diagram

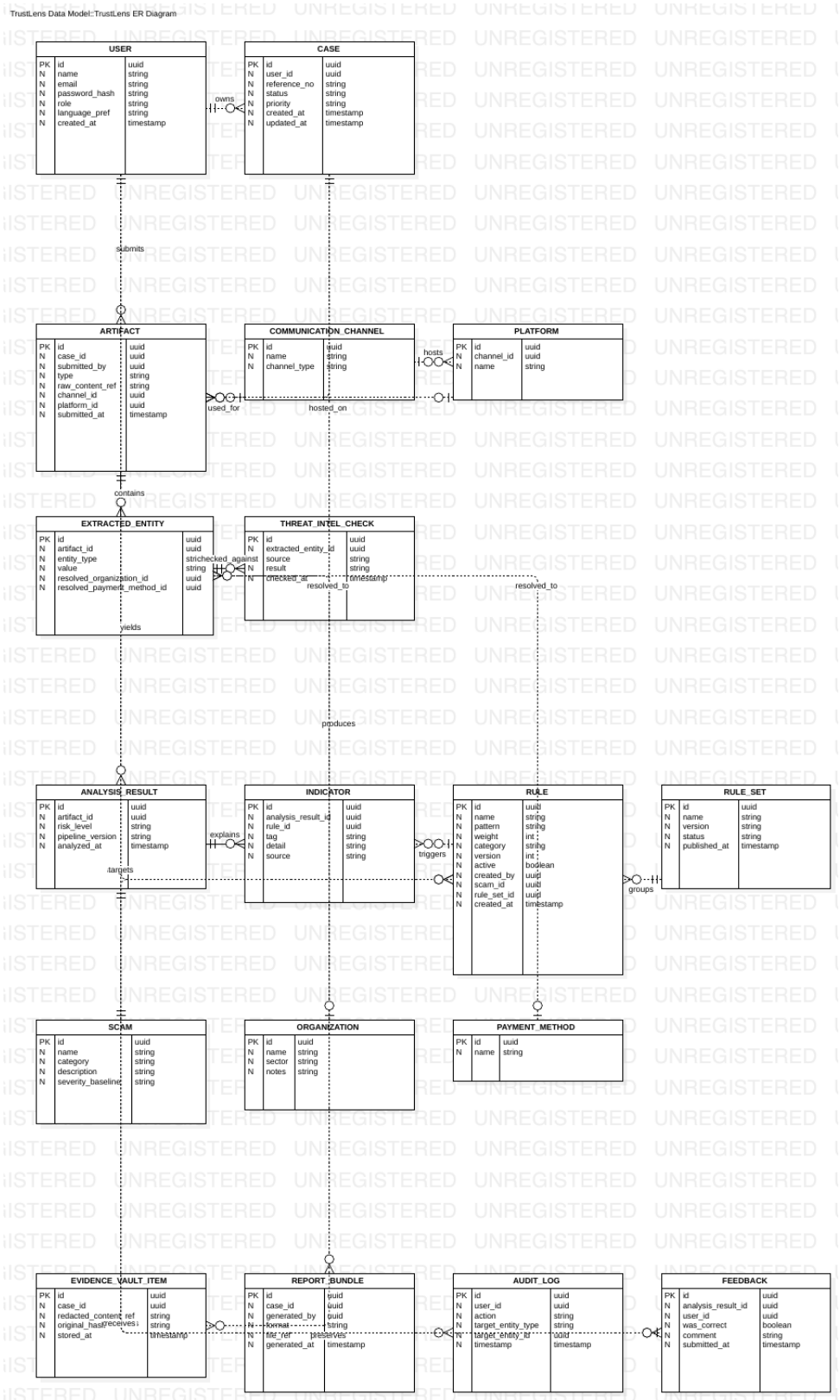


Figure B-1 — TrustLens entity relationship model

The data model is built around four groups. **Submission and evidence**: `USER`, `CASE`, `ARTIFACT`, `COMMUNICATION_CHANNEL`, `PLATFORM` and `EXTRACTED_ENTITY` hold what was submitted and what was pulled out of it. **Knowledge**: `RULE`, `RULE_SET`, `INDICATOR`, `SCAM`, `ORGANISATION` and `PAYMENT_METHOD` hold the versioned detection knowledge. **Analysis**: `ANALYSIS_RESULT` and `THREAT_INTEL_CHECK` hold what a single evaluation concluded and what any external lookup returned. **Output and governance**: `EVIDENCE_VAULT_ITEM`, `REPORT_BUNDLE`, `AUDIT_LOG` and `FEEDBACK` hold the preserved evidence, the exportable package, the append-only audit trail and the user feedback that never auto-trains anything.

The relationship that matters most for requirement 4.3.3.8 is `ANALYSIS_RESULT` to `RULE_SET`: an analysis stores the rule-set version it used, which is what makes a replay months later reproduce the same answer.

B.2 Data Flow Diagrams

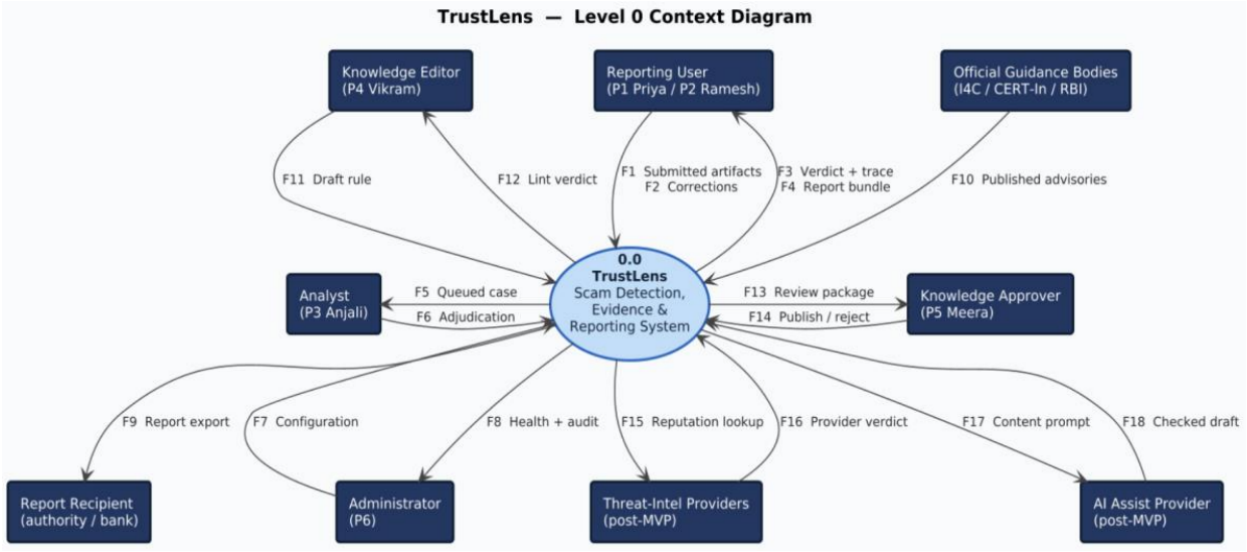


Figure B-2 — Level 0 context diagram. One process, nine external entities, eighteen flows.

The level 0 diagram fixes the product boundary. Flow F9, the report export, leaves process 0.0 rather than passing between two external entities. That is both a rule of the notation and an accurate statement of the constraint in Section 2.5.1: the export is user-initiated and access-controlled, never an automatic submission. The AI assist provider sits outside the boundary, with schema-checked drafts coming back inward, which puts constraint 2.5.3 into the structure of the diagram rather than only in prose.

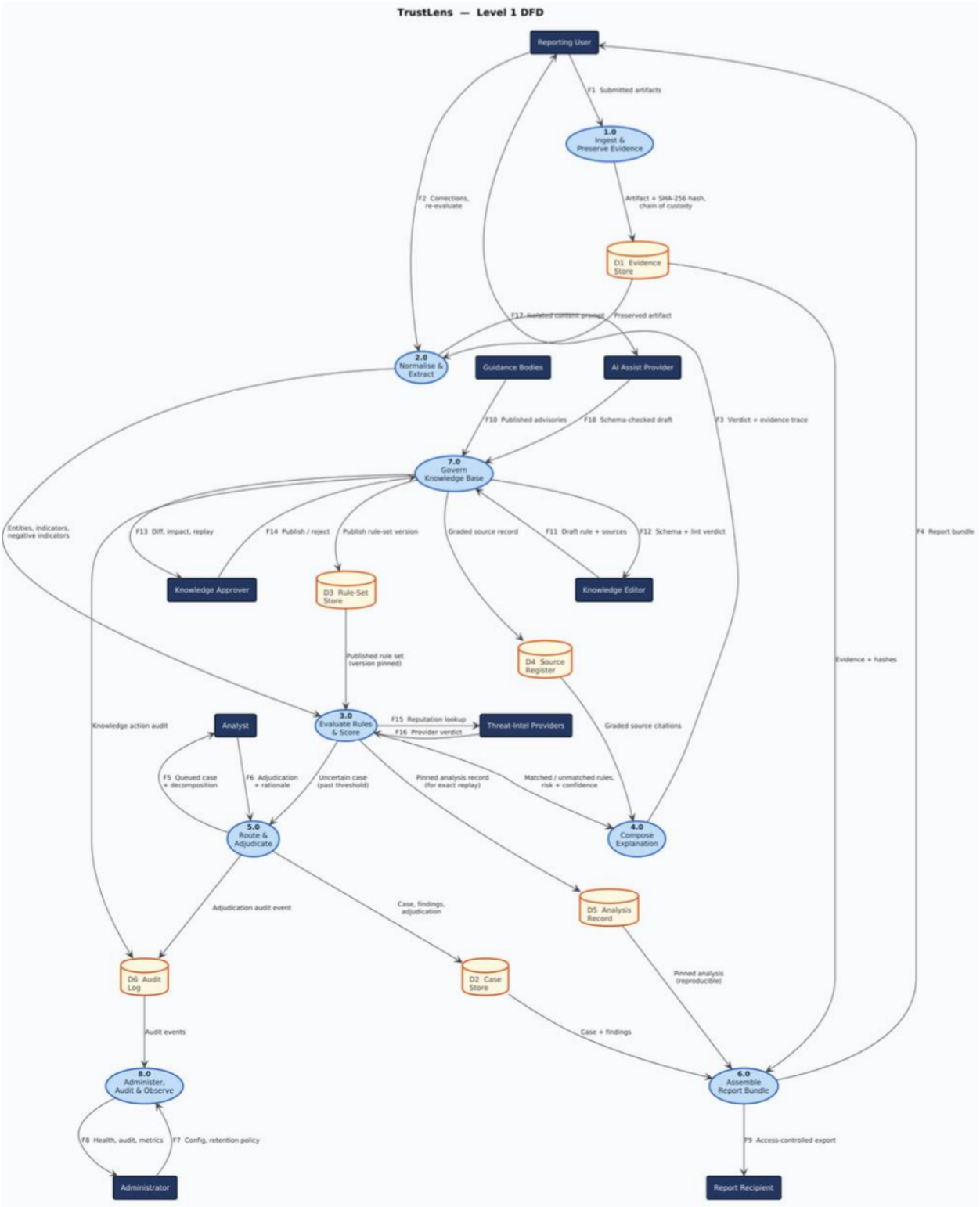


Figure B-3 — Level 1 data flow diagram. Eight processes, six data stores.

Level 1 explodes process 0.0 into the eight processes listed in Section 2.2, plus six data stores: `D1` Evidence Store, `D2` Case Store, `D3` Rule-Set Store, `D4` Source Register, `D5` Analysis Record and `D6` Audit Log.

Every flow at level 0 reappears here.

Process	Level 0 flows it inherits
`1.0` Ingest and preserve evidence	F1
`2.0` Normalise and extract	F2, F17
`3.0` Evaluate rules and score	F15, F16
`4.0` Compose explanation	F3
`5.0` Route and adjudicate	F5, F6
`6.0` Assemble report bundle	F4, F9
`7.0` Govern knowledge base	F10, F11, F12, F13, F14, F18
`8.0` Administer, audit and observe	F7, F8

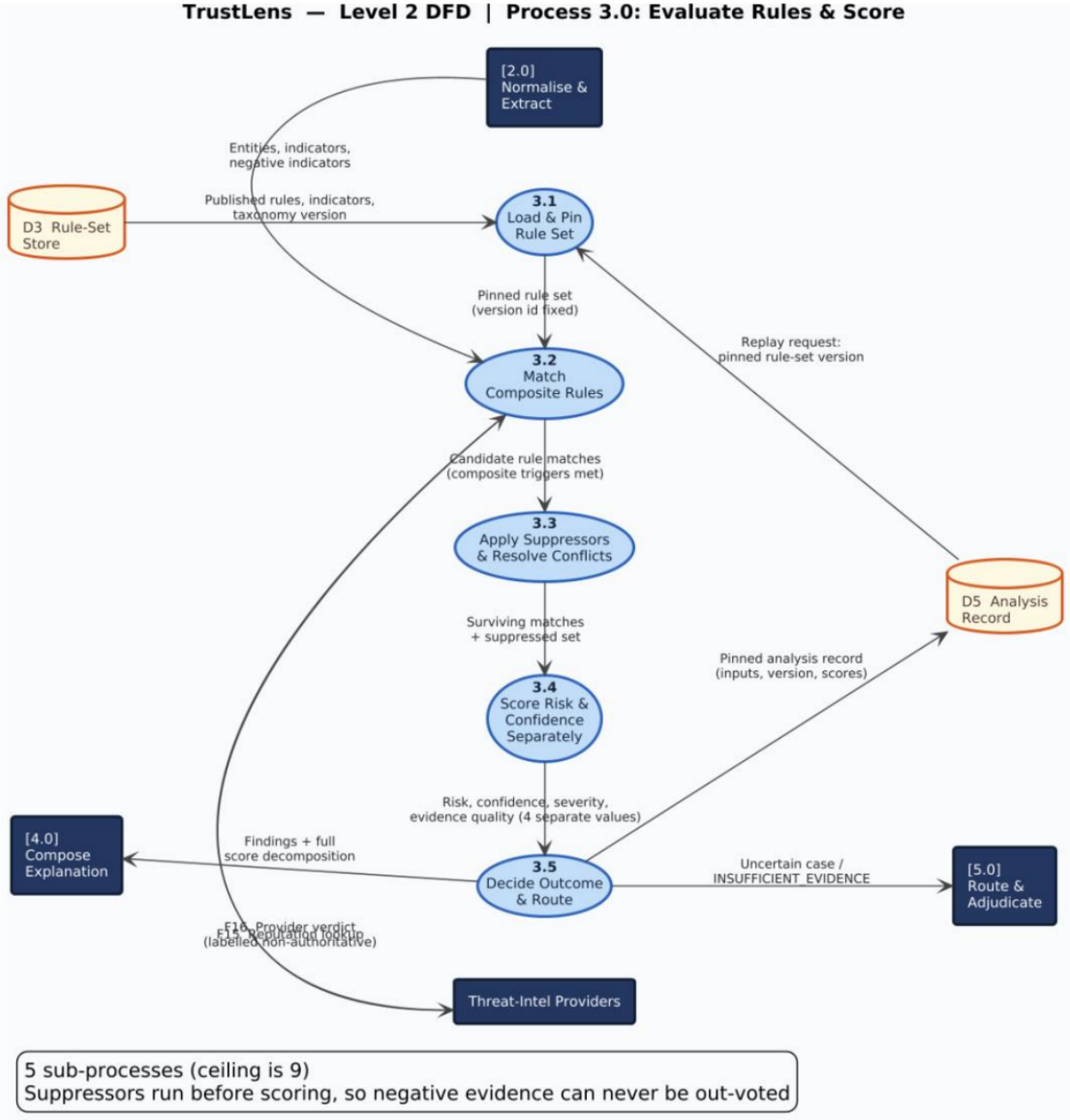


Figure B-4 — Level 2 data flow diagram, process 3.0 exploded into five sub-processes.

Level 2 breaks process 3.0 into `3.1` load and pin the rule set, `3.2` match composite rules, `3.3` apply suppressors and resolve conflicts, `3.4` score risk and confidence separately, and `3.5` decide the outcome and route it. The ordering is itself a requirement: 3.3 runs before 3.4, so negative evidence cannot be out-voted after the scoring has already happened. Process `3.5` writes the pinned analysis record that makes requirements 4.3.3.9 and 4.6.3.3 possible.

B.3 Use Case Diagram

Figure B-5 — TrustLens use case diagram

The use case diagram shows how the four actors interact with the system. The **User** authenticates and submits suspicious content as message text, a URL or a screenshot. Analysis pulls in several mandatory steps through `<<include>>` relationships: extracting entities, checking threat intelligence, applying the detection rules and computing a risk score, all before a verdict is shown. The user can then view the matched indicators, preserve the evidence, generate a report bundle as PDF or JSON, manage the case, and submit feedback.

Optional behaviour is drawn with `<<extend>>`: escalating a case to an analyst, and submitting feedback after viewing a verdict. The **Security Analyst** reviews flagged cases and adjudicates findings. The **Administrator** manages the detection rule lifecycle by creating, approving and publishing rule sets. The **Threat Intelligence Provider** is an external actor that supplies the reputation data the detection process can

draw on.

Two things are deliberately absent from this diagram, and both are constraints rather than oversights. There is no "submit report to authority" use case, because the product never does that (Section 2.5.1). And there is no actor that can publish a rule without going through approval, because publication needs the approver role (business rule 5.5.5).

Appendix C: To Be Determined List

These are the things that are genuinely still open. Nothing in this list is quietly treated as settled elsewhere in the document.

ID	Item	What it blocks	Needed from
C.1	No end user, analyst or knowledge editor has validated any persona or journey.	Confidence in Section 2.3 and much of Section 4	Access to even one real user
C.2	Data retention period and legal basis are undecided, and DPDP applicability is unverified.	4.8.3.4, 4.8.3.5, 5.3.3, Section 6	Sponsor decision plus a qualified legal review
C.3	Docker and PostgreSQL are not installed on the development machine.	All implementation	An administrative install
C.4	Which Indian languages are in release 1.0 scope. Currently English only, and no verified source gives us non-English cues.	4.2.3.3, 5.2.6, Section 6	Sponsor decision
C.5	Default evidence retention is 90 days as a placeholder, not a policy decision.	4.8.3.5	Sponsor decision
C.6	Deployment target unknown, local only or hosted.	Section 2.4 and the operational requirements	Sponsor decision
C.7	Identity provider not selected.	4.8.3.1	Architecture decision, later phase
C.8	OCR engine not selected, and there is no guidance on OCR quality for Indian scripts.	4.2.3.4, 5.1.2	Empirical evaluation
C.9	The negative-indicator library does not exist yet. The research package gave us zero suppressive signals.	4.2.3.7, 4.3.3.3, and therefore the whole false-positive strategy	Knowledge work package
C.10	The reduction maths for `REDUCE`-type suppression rules is undefined. Only `SUPPRESS` is currently expressible.	4.3.3.3, 4.3.3.10	Detection design phase
C.11	Seven I4C-attributed rule bases and three commercial-source rule bases could not be retrieved, so the rules resting on them stay unpublished.	Detection coverage at release	Manual retrieval on an Indian network
C.12	No labelled real-world corpus exists, so accuracy is unmeasurable. This one cannot be closed within this project and is recorded so it is not quietly forgotten.	Section 5.4.2 and any accuracy claim	Out of scope. Permanent disclosure required.
C.13	Sextortion appears in Chakshu's official reporting categories but is missing from our scam taxonomy.	Taxonomy completeness	Knowledge work package
C.14	Loan-app abuse and mule-account categories currently have zero cited sources.	Detection coverage	A dedicated research pass
C.15	The project has no end date, so no requirement in this document is time-boxed.	Release planning	Sponsor decision

End of SRS-001 v2.0.